

Reconstrucción de escenas mediante Reversión Temporal Acústica (ATR)

Justificación

Tradicionalmente, las ciencias naturales utilizan el estado presente de un sistema para predecir su evolución futura. Sin embargo, en la investigación forense y la acústica aplicada, surge el reto del *Problema Inverso*: utilizar datos capturados en el presente para deducir con exactitud el origen de un evento. Esta propuesta plantea invertir la «flecha del tiempo» mediante la técnica de **Reversión Temporal Acústica (ATR)**. Al explotar la simetría temporal de la ecuación de onda, el software propuesto procesará señales acústicas grabadas para retroceder numéricamente hasta el instante inicial. El objetivo es transformar el ruido capturado por sensores en una herramienta de deducción científica capaz de localizar fuentes sonoras y reconstruir escenas con una precisión superior a los métodos convencionales.

Metodología

Para resolver el problema, se partió desde la ecuación de presión acústica $u(x, y, t)$:

$$\nabla^2 u - \frac{1}{v^2} \frac{\partial^2 u}{\partial t^2} = 0 \quad (1)$$

En donde si se separan variables con u , la ecuación queda en términos espaciales y temporales. La parte espacial se solucionó con la transformada de Fourier y la parte temporal se solucionó con Leapfrog.

Código de C++

`malla.alternativa-cpp` comienza definiendo una clase permite encapsular tanto los datos como la lógica necesaria para construir la geometría de la habitación. De esta forma, por medio `Habitacion` el constructor calcula automáticamente el espaciamiento de la malla y la ubicación de los sensores, evitando que estos cálculos deban realizarse de manera externa y favoreciendo una implementación más modular y eficiente.

Falta explicar paralelización. Posterior a la creación del espacio en el que se trabajará; la función `ejecutar_solver_atr_optimizado` es la que realiza la simulación. Esta se hace con archivos binarios para evitar tener que guardar muchos archivos `.csv`. La variable `dt` es simplemente una condición CFL que aporta estabilidad al método.

Con esto, el código prepara todo para comenzar con las transformadas donde cada variable definida contendrá un

aspecto importante: en primer lugar de crean tres arreglos de tamaño $N_x N_y$ donde cada una almacena los momentos acústicos (`u_pasado` = u^{n-1} , `u_presente` = u^n , `u_futuro` = u^{n+1}). `u_hat` contendrá $\hat{u}$, que se define por medio de $\mathcal{F}\{u(x, y)\} = \hat{u}$, `lap_hat` tendrá $-k^2 \hat{u}$ y `lap_real` contendrá $\nabla^2 u$. Después se construyen los vectores k_x y k_y , que da lugar a la igualdad de $k^2 = k_x^2 + k_y^2$. Donde k representan los números de onda.

Luego, la parte activa del algoritmo es el ciclo `for` (`int n = 0; n < Nt - 1; n++`) realiza el cálculo de $\nabla^2 u$ desde el inicio de la simulación mandándolo al espacio de Fourier. Lo primero que hace con `fftw_execute(plan_directo)`; es hacer $\mathcal{F}\{u(x, y)\} = \hat{u}(k_x, k_y)$ y lo almacena en la variable respectiva. Luego una de las partes que se paralelizan es el cálculo de $\mathcal{F}\{\nabla^2 u\} = -k^2 \hat{u}$, pero al ya tener $\hat{u}$, lo que se hace es multiplicar por $-k^2$. Por último; `fftw_execute(plan_inverso)`; aplica Fourier inversa $\mathcal{F}^{-1}\{-k^2 \hat{u}\} = \nabla^2 u$ almacenando este resultado es su respectiva variable y actualiza la ecuación de ondas con la laplaciano que ya se conoce. `u_futuro[k] = 2.0 * u_presente[k] - u_pasado[k] + (v * v * dt * dt) * lap_normalizado`; queda como

$$u^{n+1} = 2u^n - u^{n-1} + v^2(\Delta t)^2 \nabla^2 u \quad (2)$$

en donde sale partiendo de (1) y aplicando diferencias finitas para el cálculo de las derivadas parciales. Específicamente aplicando el método de *leapfrog*, que soluciona la parte temporal.

Posterior a esto, se realiza un corrimiento temporal donde el futuro pasa a ser el nuevo presente y el presente pasa a ser el nuevo pasado, para luego cerrar el archivo binario que se ha venido escribiendo durante toda esta inversión temporal y lo guarda en un archivo `.csv` y escribe una matriz reconstruida, que es la que leerá el Python.

Por último, borra las variables que se definieron anteriormente, pues esto es un buen hábito para evitar *memory leaks* y en la función `main` se define la dimensión original de la malla y la cantidad de pasos temporales.

Paralelización de C++

El algoritmo se paralelizó mediante OpenMP, empleando directivas `pragma omp parallel for` en las partes repetitivas del algoritmo en donde no hubieras dependencias temporales. La primera parte que se paralelizó fue la copia del campo a la entrada de FFTW, que depende enteramente de la dimensión de la malla, que es este caso se estableció de 128×128 , en donde los hilos se reparten este trabajo, puesto que no importa el orden en que se haga la actualización.

Asimismo, el cálculo del operador espectral $-k^2\hat{u}$, en donde como cada punto en el espacio de Fourier es independiente del otro, los hilos pueden optimizarlo. También en la parte de la ecuación (2) que se hizo con *leapfrog*, donde entre los hilos se reparten el cálculo de los nodos a futuro y se evalúa de manera independiente para cada punto de la grilla. Y finalmente, lo último que se paralelizó fue la actualización de los arreglos, donde se reparten distintas partes de este.

Es importante tomar en cuenta que las partes del código en donde hay que actualizar los nodos a futuro donde depende del anterior no pueden paralelizarse. Como en la parte `for (int n = 0; n < Nt-1; n++)` (del que se habló anteriormente). El instante $n + 1$ depende del instante anterior n .

Todos estos procesos tienen en común que cada hilo realiza la misma tarea, pero desde distintos puntos de inicio en la malla. De la misma forma en la que la suma de vectores de puede paralelizar si entre los hilos se reparten las tareas dependiendo enteramente de cuántas componentes tenga este.

Código de Python

Python se empleó para la graficación de los mapas de calor tanto en 2D como en 3D.

Del archivo `visualizador_an.py`, la función `antes_regresion` genera la figura 2, la cual corresponde a la animación de la onda de choque en el centro de la habitación, distribuyéndose uniformemente hasta chocar con los sensores en los extremos de esta.

Luego, en orden; la función `generar_reconstruccion` genera la figura 1, al cual corresponde al momento final de la regresión temporal, donde el sonido termina de converger en el punto inicial.

Posterior a esta; la función `graficar_malla_3d`, hace lo mismo que la función anterior, pero graficado a 3 dimensiones, es decir; la figura 4. Es por esto que tanto 1 como 4 corresponden al mismo «entorno», pero aumentando la altura de los valores de la presión.

La cuarta función; `animar_malla` genera uno de los mapas de calor animado. (el que corresponde a `viaje_del_sonido.mp4` en [1]).

Y por último; la función `animar_reconstruccion` genera el otro video, que corresponde a la regresión temporal animada (el archivo llamado `inversion_temporal.mp4` en [1]).

Resultados

Pero al final, ¿qué se obtuvo?

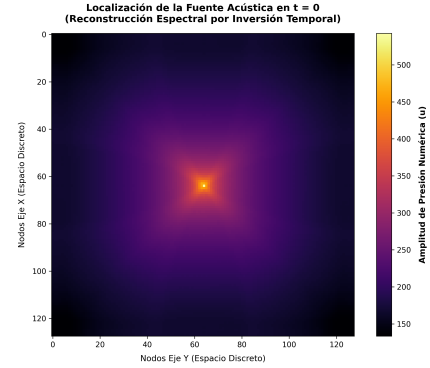


Figura 1: Heatmap del instante después de la regresión temporal

Este primer *heatmap* representa el punto en donde se da la mayor amplitud de presión. Lo cual tiene sentido considerando que se colocó el origen del sonido en el centro porque se necesita una posición arbitraria para tener el fenómeno de estudio.

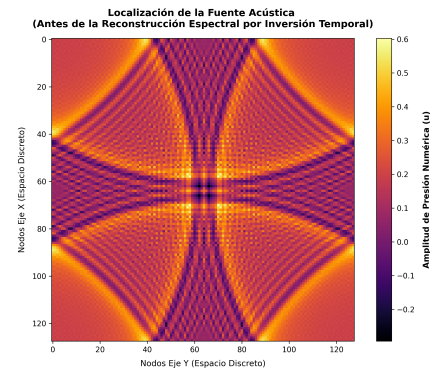


Figura 2: *Frame* de la primera animación

Esta segunda figura corresponde a un *frame* de la animación de `viaje_de_sonido.mp4`, el cual como ya se explicó previamente, anima el proceso de la onda sonora desde el centro de la habitación hasta que colisiona con los sensores colocados en los extremos de la malla.

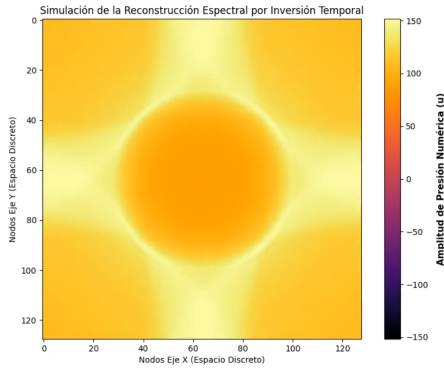


Figura 3: *Frame* de la segunda animación

Esta figura representa un *frame* de la animación `inversion_temporal.mp4`, el cual es la regresión de la onda sonora hasta el punto de convergencia, es decir; al origen del sonido.

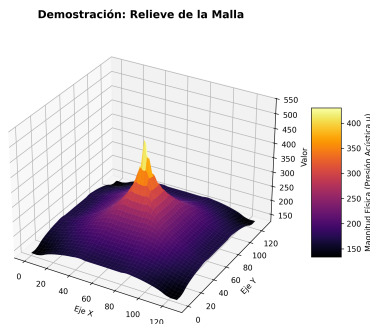


Figura 4: Visualización de la malla en 3D

Y por último; esta figura en 3D, puede verse que es muy similar a 1, pero aumentando la presión en el punto inicial a una altura para una visualización en 3 dimensiones.

Fallas de la simulación

La parte del código en la que puede provocarse la falla viene dada por:

```
const double termino_especial = std::sqrt((1.0 / (sala.hx * sala.hx)) + (1.0 / (sala.hy * sala.hy)));
const double dt = (2.0 / (M_PI * v * termino_especial)) * 0.9;
```

Figura 5: Código que propicia la falla

A la hora de hacer pruebas con la condición del número de Courant, descubrimos que el argumento (que en 5 es 0,9), que funciona como «factor de seguridad» debe ser menor o igual a 1. En caso contrario, el programa se vuelve inestable. Pero, ¿por qué sucede esto? El número de Courant determina la cantidad de información en una celda de la malla que

se propaga en una unidad de tiempo. Al aumentar este factor de seguridad a un número mayor a 1, se propaga más información de la que se puede leer, por lo que el algoritmo se vuelve inestable.

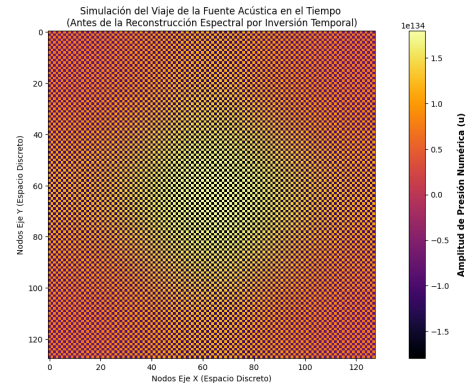


Figura 6: *Frame* de la animación de la falla

La animación evidenciando esta falla, al igual que los anteriores, puede verse en [1].

Referencias

- [1] D. Mondragón, “Proyecto-atr.” <https://github.com/dmondra/Proyecto-ATR>, 2026. Repositorio de GitHub.
- [2] J. Lorente, “Capítulo 5: Método de diferencias finitas y número de courant,” n.d.
- [3] Ideal Simulations, “Courant number in cfd,” n.d.
- [4] K. E. Niemeyer, “Parabolic partial differential equations,” n.d.
- [5] CFD Land, “What is a courant number (cfl number)?,” n.d.
- [6] Matplotlib Development Team, “Animations using matplotlib,” n.d.
- [7] Stack Overflow, “How to use matplotlib funcanimation to animate a heatmap,” 2020.
- [8] C. Canuto, M. Y. Hussaini, A. Quarteroni, and T. A. Zang, *Spectral Methods: Fundamentals in Single Domains*. Springer, 1 ed., 2006.
- [9] M. Fink, “Time reversal of ultrasonic fields. i. basic principles,” *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control*, vol. 39, no. 5, pp. 555–566, 1992.